HCL DX Sizing / Estimation Tool

User Guide — Estimator, Simulator, Calibration & Live Compare

App path: `DXAutomation/dx-sizing-tool` **Run:** `npm run dev` → http://127.0.0.1:5173/

Profiles: dxcn-cf238 · compose-cf237 **Version:** 0.1.0

> **Disclaimer.** This is an estimation aid for lab and planning use. It is *not* an HCL official SCU, sizing guarantee, or license quote. Calibrate capacity profiles on your own environment before production decisions.

1. Contents

1. What this tool does
2. Start the app
3. UI layout
4. DXCN vs DX Compose
5. Inputs (must-have & advanced)
6. Estimator
7. Simulator
8. Calibrate (Locust → profile)
9. Live vs estimate
0. Exports
1. How the math works
2. Worked example
3. Troubleshooting

2. What this tool does

Given business/usage inputs (peak users, session length, search/DAM attach rates, etc.), the tool produces two views from the **same sizing engine**:

Estimator

BOM table: replicas, CPU, memory, storage, util, infra cost, license cost, AI tokens. Session panel + CSV/JSON/Helm export.

Simulator

Topology graph with RPS on edges, util coloring, load slider (0.1×–5×), scenario presets, and fix actions (add replica / enable CDN).

Switching product **DXCN ↔ DX Compose** swaps the component catalog (Core vs webEngine; Remote Search only on DXCN) but keeps one math kernel so estimator and simulator never disagree.

3. Start the app

```
cd DXAutomation/dx-sizing-tool
npm install
npm run dev
# open http://127.0.0.1:5173/

npm test          # engine unit tests
npm run build     # production bundle → dist/
```

Settings persist in browser `localStorage` key `dx-sizing-tool-v1` (product, inputs, toggles, unit costs).

4. UI layout

Area	Purpose
Top bar	Product switch (DXCN | Compose) and tabs (Estimator | Simulator | Calibrate | Live vs est)
Left sidebar	Scenario presets, must-have metrics, advanced accordion, optional component toggles, unit costs
Main panel	Active tab output (BOM, graph, calibration, live compare)

5. DXCN vs DX Compose

	DXCN	DX Compose
Portal runtime	DX Core	Web Engine
Classic Remote Search	Optional	Not available (toggle forced off)
Search V2 (MW + OpenSearch + file processor)	Optional	Optional (common on labs)
Always-on	HAProxy, logging sidecar, runtime controller, prereqs (job-only), external DB	
Optionals (shared)	OpenLDAP, External IdP, Content Composer, Ring API, DAM + image processor + persistence*, plugins, License Manager, People Service, IQ + MCP	

> **Toggle forcing.** Enabling DAM also enables Ring API, Image Processor, and persistence pool/node/metrics. Content Composer forces Ring API. Any Search V2 piece enables the full trio. IQ Integrator forces MCP.

> **Auth topology.** External IdP (OIDC/SAML) is *not* reached via HAProxy. The browser redirects to the IdP; Core / webEngine call the IdP back-channel (token, JWKS, etc.). OpenLDAP binds are also Core/webEngine → LDAP directly. ACS/callback traffic returns as normal portal RPS through HAProxy.

6. Inputs

6.1 Must-have

Input	Why it matters
Peak concurrent users	Session memory + derived RPS
DAU / MAU	Sessions/day, storage & token projections
Peak RPS override	If > 0, replaces derived front-door RPS
Req / session & session duration	`RPS ≈ U_peak × req / (T_min × 60)`
% anonymous / authenticated	Login share → LDAP binds / IdP browser redirect + Core back-channel (not HAProxy→IdP)
% browse / author / form	WCM vs authoring vs write load
Search / DAM attach rates	Fan-out to search & DAM paths
Assets / page	DAM RPS multiplier
AZ count	HA replica floor
Target p95 (ms)	Documented SLO (headroom context)
Cache hit ratio	Largest lever — reduces portal RPS after CDN

6.2 Advanced

Think time, searches/hit, peak authoring users, uploads/day & avg MB, content items (index GB), IQ attach / msgs / tokens in+out, growth CAGR, session footprint KB.

6.3 Scenario presets

Preset	Effect
Anonymous browse	Mostly anon portal traffic, light search/DAM
Login + WCM	Authenticated mix with form posts
Authoring	Raises author %, enables Content Composer + Ring
DAM heavy	High attach/assets/uploads; enables DAM stack
Search heavy	High search attach; Remote Search (DXCN) or Search V2 (Compose)
IQ chat	Enables IQ + MCP; token cost path

6.4 Unit costs

Editable $/vCPU-hour, $/GiB-hour, $/GiB storage-month, $/1K tokens, DX license $/node-month, self-host LLM RAM GiB. Infra and license totals are kept separate on purpose.

7. Estimator

Summary cards show front-door RPS, portal RPS, concurrent sessions, infra $, license $, AI tokens $, total $, and profile confidence.

- **Session math** — concurrent sessions, sessions/day, footprint Mi, recommended timeout, sticky-session note for HAProxy.
- **BOM table** — per component: replicas, offered/capacity RPS, util, CPU, mem, storage, infra $, license $.
- **Util colors** — ● green <70% · ● amber 70–90% · ● red >90%.
- **Warnings** — default-profile confidence, CDN miss=0, bottlenecks, Compose Remote Search ignored, etc.

8. Simulator

1. Open the **Simulator** tab. Graph auto-layouts from the active product topology.
2. Edge labels show traffic class + RPS; edges animate when RPS > 0.
3. Use the **Load** slider (0.1×–5×) to stress the design. At ~3× peak you should see amber/red on hot path nodes.
4. Click a node for util, replicas, cost, and notes.
5. **Add replica** raises that node's replica override and recomputes util/cost.
6. **Enable CDN** restores cache-hit path (uncheck CDN to force miss-through).
7. **Reset fixes** clears overrides and sets load back to 1×.

> Simulator and Estimator call the same `runSizing()` function. Node panel numbers match the BOM for identical inputs (except temporary replica overrides / CDN toggle in the sim).

9. Calibrate (Locust → profile)

Default profiles are conservative placeholders. To make numbers trustworthy on your lab:

1. Run Locust scenarios; measure steady-state RPS and CPU per component (and replica count).
2. Open **Calibrate**. Paste JSON like:

```
{
  "measurements": [
    { "componentId": "core", "rps": 140, "cpuCores": 6, "replicas": 2, "util": 0.7 },
    { "componentId": "haproxy", "rps": 800, "cpuCores": 0.4, "replicas": 2, "util": 0.5 }
  ]
}
```

3. Click **Apply calibration**. A lab profile is registered (e.g. `dxcn-cf238-lab`) with `confidence: lab-calibrated`.
4. Optionally **Export Locust scenario mix** (browse/author/form/search/DAM/IQ weights) for AIOps alignment.
5. Sample file: `scripts/sample-locust-measurements.json`.

Patch formula (approx): `rpsPerReplica ≈ observed_rps / replicas / util`; `cpuPerRps` derived from measured CPU above base.

10. Live vs estimate

1. Capture `kubectl top pods` (or paste an AIOps-style summary in the same columns).
2. Open **Live vs est**, paste, click **Compare vs estimate**.
3. Pod names are matched by substring map (`dx-core` →core, `haproxy`, `web-engine`, `dam`, …).
4. Status: **ok** · **over** (live ≫ estimate) · **under** (live ≪ estimate) · **missing** (no match).

11. Exports

Export	Where	Use
JSON	Estimator	Full sizing result (demand, nodes, costs, helmHints)
CSV	Estimator	BOM spreadsheet
Helm resource hints	Estimator / Calibrate	YAML-ish `scaling` + `resources` requests/limits — starting point only
Copy Helm hints	Estimator	Clipboard
Locust scenario mix	Calibrate	Task-weight JSON for AIOps Locust

12. How the math works (short)

```
frontDoorRps = peakRpsOverride OR (U_peak × req_per_session / (T_session_min × 60))
               × loadMultiplier × (1 + CAGR)

portalRps    = frontDoorRps × (1 − cacheHitRatio)

searchRps    = portalRps × searchAttach × searchesPerHit   (if search enabled)
damRps       = portalRps × damAttach × assetsPerPage       (if DAM enabled)
imageRps     = 0.2 × damRps
iqRps        = U_peak × iqAttach × msgs / sessionSec       (own traffic class)

replicas     = max( ceil(offeredRps / rpsPerReplica), haMin, azCount, minReplicas )
util         = offeredRps / (replicas × rpsPerReplica)

mem_sessions (Core/webEngine) = U_peak × sessionBytes × 1.3 safety
tokens/mo    = sessions/day × 30 × iqAttach × msgs × (tok_in + tok_out)
DAM storage  ≈ uploads/day × avgMB × 30 × retention / 1024  Gi
```

Capacity constants live in `src/profiles/dxcn-cf238.json` and `src/profiles/compose-cf237.json` (`rpsPerReplica`, `cpuPerRps`, `memBaseGi`, HA floors, etc.).

13. Worked example

Scenario: 5,000 peak concurrent users · 40% search attach · 10% DAM · IQ off · DXCN · cache hit 0.4

1. Select **DXCN**. Leave defaults (or set peak users = 5000, search attach = 0.4, DAM attach = 0.1).
2. Enable **Remote Search** and **DAM** (DAM auto-enables Ring + image + persistence).
3. Estimator shows Core ≥ 2 replicas (HA floor), Remote Search and DAM sized from fan-out RPS, separate infra vs license totals.
4. Open Simulator → slide load to **3×**. Expect Core (and possibly search/DAM) to move amber/red.
5. Click the red node → **Add replica** until util drops under 70%, or raise cache hit / lower attach rates.

14. Troubleshooting

Symptom	Fix
npm can't find package.json	Run from `dx-sizing-tool/`, not the repo root
Remote Search on Compose	Expected — Compose forces it off; use Search V2
Always 2+ Core/webEngine replicas	HA/AZ floor by design (never recommend 1 for prod)
Huge portal RPS	Raise cache hit ratio; or set peak RPS override explicitly
Live compare all "missing"	Pod names don't match map — rename needles or include `core` / `haproxy` / `dam` substrings
Stale UI state	Clear `localStorage` key `dx-sizing-tool-v1`

15. Project map

```
dx-sizing-tool/
  src/engine/          # pure TS kernel (demand, size, runSizing, helm, calibrate, live)
  src/catalogs/        # DXCN + Compose topology + toggle resolve
  src/profiles/        # CF-tagged capacity JSON
  src/components/      # Estimator, Simulator, Calibrate, Live panels
  scripts/             # sample Locust measurement JSON
  docs/user-guide.html # this guide (source)
```